AIStudio

STD - AIStudio - Codebase Guide - 2026-04-28

Type: STD | Domain: AIStudio | Status: ACTIVE Version: 1.1.0 | Created: 2026-04-06 | Last updated: 2026-04-30 | Owner: Manuel Barbero

Purpose

This document provides a structured introduction to the AIStudio codebase — for a new developer, a technical reviewer, or an AI agent bootstrapping context. It describes the general architecture, the key decisions made, and the first two layers of the directory hierarchy. It does not describe every file in detail; that is what the code itself is for.

General Principles

AIStudio follows conventional Python project structure with deliberate discipline around a few non-standard choices:

Standard practices adhered to: - `src/` layout — all application code lives under `src/`, keeping the repo root clean - `pyproject.toml` as the single project config file (ruff, pytest settings) - `tests/` at repo root — test discovery is unambiguous - Pre-commit hooks (ruff lint + format) enforced on every commit - GitHub Actions CI runs the full test suite on every push - Conventional commits in practice

Non-standard decisions (intentional): - A single HTML file (`front_end/rag_studio.html`) is the entire frontend — no build step, no node_modules, no bundler. All JS and CSS are inline. AIStudio runs fully offline and the UI must be self-contained. - `data/corpora/` is partially tracked — the shipped `demo/` corpus is tracked in git. The `help/` corpus config file (`help_corpus_metadata.yaml`) is tracked; its PDFs are regenerated at startup. User-generated corpora are gitignored. - No ORM, no database — the application uses Qdrant (vector store) and flat JSONL files for corpus metadata. The data model is intentionally simple for a local,

single-user application. - `ais_install` is manifest-driven — `bundle_manifest.yaml` is the single source of truth for command paths and installation type. Adding a new command requires a manifest entry; `ais_install [cmd]` then installs it without touching other aliases.

Architecture Overview

AIStudio is a local RAG (Retrieval-Augmented Generation) application for Apple Silicon. The architecture has three layers:

Ingest layer — documents are loaded, chunked, embedded, and stored in Qdrant. This is a one-time operation per document, triggered via the UI. The ingest pipeline lives in `src/local_llm_bot/app/ingest/`.

Query layer — a FastAPI backend receives questions, retrieves relevant chunks from Qdrant, reranks them with a CrossEncoder, assembles a prompt, and sends it to an Ollama-hosted LLM. Citations are extracted from the response and returned to the frontend. The query pipeline lives in `src/local_llm_bot/app/rag_core.py` and is orchestrated by `api.py`.

UI layer — a single HTML file provides the complete user interface: corpus management, file upload, chat, settings, and citations rendering. It communicates with the FastAPI backend over localhost.

The three services that must be running are: **Qdrant** (vector store), **Ollama** (LLM host), and the **FastAPI backend** (uvicorn). `ais_start` starts all three.

First-Level Directory Structure

```
AIStudio/
├── src/            Application source code
├── tests/          Test suite
├── front_end/      Single-file frontend (rag_studio.html)
├── data/           Corpus data (partially tracked)
├── docs/           Developer documentation and help corpus sources
├── benchmarks/     Benchmark harness and reports
├── prompts/        LLM system prompt
└── [root files]    Install scripts, user aliases, README, HOWTO, config
```

Second-Level Structure

`src/local_llm_bot/app/` — Core Application

Path	What it is
`api.py`	FastAPI application — all HTTP endpoints: `/ask`, `/corpus/*` (create, rename, delete, info, upload, ingest), `/health`, `/debug/*`, `/source`, `/prewarm`
`rag_core.py`	RAG query pipeline — embed query → Qdrant retrieve → CrossEncoder rerank → Ollama generate → extract citations
`config.py`	Application config — RagConfig, IngestConfig, OllamaConfig, loaded from environment variables
`ollama_client.py`	HTTP client wrapper for the Ollama API
`debug_stats.py`	JSONL stats computation for the `/debug/stats` endpoint
`ingest/`	Ingest pipeline — loaders, chunking, embedding, Qdrant write
`vectorstore/`	Vectorstore adapters — `qdrant_store.py` (active)
`utils/`	Shared utilities — `corpus_paths.py` (artifact paths), `repo_root.py` (repo discovery)

`src/local_llm_bot/app/ingest/` — Ingest Pipeline

Path	What it is
`__main__.py`	Entry point — `python -m local_llm_bot.app.ingest`
`pipeline.py`	Orchestrates: load → chunk → embed → store
`loaders.py`	File loaders — PDF (pdfplumber), HTM, DOCX, XLSX, MD
`chunking.py`	Semantic chunking — page-aware, respects sentence/paragraph boundaries
`types.py`	Shared types — `Document`, `Chunk`, `RetrievedDoc`
`index_jsonl.py`	`index.jsonl` read/write — chunk metadata on disk
`manifest.py`	`manifest.jsonl` tracking — which files have been ingested

`data/corpora/<name>/` — Corpus Structure

Each corpus has a consistent on-disk layout:

Path	What it is
`uploads/`	Source files uploaded by the user
`trash/`	Files removed from corpus (recoverable) — sibling of uploads/, never inside it
`index.jsonl`	Chunk metadata index
`manifest.jsonl`	File tracking manifest
`doc_chunk_map.json`	Maps documents to their chunk IDs
`{name}_corpus_metadata.yaml`	Search routing guidance — loaded into system prompt at query time

Tracked in git: Only `data/corpora/demo/` (full uploads tracked — ships with repo) and `data/corpora/help/help_corpus_metadata.yaml` (config only — PDFs are regenerated by `ais_update_help_ops`). All other corpus data is gitignored.

`docs/` — Documentation

User-facing and developer documentation. All docs with a PDF companion are regenerated via `ais_update_corpus_ops help` and ingested into the `help` corpus.

File	What it is
`architecture_decisions.md/pdf`	Architecture Decision Records — Qdrant, CrossEncoder, chunking, citation design
`DEMO_CORPUS.md/pdf`	Demo corpus content description
`HARNESS.md/pdf`	Benchmark harness documentation
`PRODUCT_ROADMAP.md/pdf`	PM-facing roadmap — Beta, v2.0, v3.0
`QA_TESTING_LESSONS_LEARNED.md/pdf`	Install and QA testing findings
`dependencies.md/pdf`	Python and system dependency list
`CODEBASE_GUIDE.md/pdf`	This document — codebase introduction

`[root files]` — Install Scripts and User Commands

File	What it is
`ais_install`	Manifest-driven user command installer — `ais_install [cmd]` installs one alias; `ais_install` installs all
`ais_install_ops`	Manifest-driven operator command installer — mirrors `ais_install` for operator commands
`install.sh`	First-time bootstrap — sets up Python venv, installs deps, calls `ais_install`
`install_ops`	First-time operator bootstrap — calls `ais_install_ops`
`ais_start.sh`	Start all services (Qdrant, Ollama, FastAPI, opens UI) — aliased as `ais_start`
`ais_stop.sh`	Stop all services — aliased as `ais_stop`
`ais_bench.sh`	Run benchmark on demo corpus
`ais_log.sh`	Tail live backend log — aliased as `ais_log`
`ais_download_sec_10k.sh`	Download SEC 10-K corpus from EDGAR
`ais_ingest_sec_10k.sh`	Ingest SEC 10-K corpus into AIStudio
`ais_help.sh`	Show user command reference

All `ais_*` commands are registered in `meta/bundle_manifest.yaml` with `alias` and `install` fields. `ais_install` reads this manifest to generate `~/.zshrc` aliases — the manifest is the single source of truth for command routing and installation. See `STD - AIStudio - Command Development and Management` for the full lifecycle standard.

Key Design Decisions

Why a single HTML file for the frontend? AIStudio runs fully offline. A build step would add complexity and a node_modules dependency. A single file can be opened directly from disk and requires no server to serve static assets.

Why Qdrant over ChromaDB? ChromaDB was the original vectorstore. Qdrant was chosen for its stability on Apple Silicon, its clean REST API, its WAL-based durability, and its superior performance at scale. The migration is complete; ChromaDB code is legacy and unused.

Why CrossEncoder reranking? Embedding-based retrieval is fast but imprecise — it retrieves semantically similar chunks, not necessarily the most relevant ones. A CrossEncoder (ms-marco-MiniLM-L-6-v2) reranks the top-K retrieved chunks against the query, improving answer quality significantly with minimal latency overhead.

Why `{corpus_name}_corpus_metadata.yaml`? Each corpus has different content and requires different retrieval guidance. The corpus meta YAML is loaded into the system prompt at query time, allowing per-corpus routing instructions without code changes.

User / Operator Command Boundary

AIStudio commands are split into two tiers with a hard boundary between them.

User commands (`ais_*`) are git-tracked, ship with the public repo, installed by `ais_install`, documented in `ais_command_help.txt`, and visible via `ais_help`. These are the commands any developer who clones the repo can use. The full list is in `ais_user_commands_manifest.yaml` at repo root.

Operator commands (`ais_*_ops`) are gitignored, travel via BUNDLE only, installed by `ais_install_ops`, documented in `ais_command_help_ops.txt` only, and never referenced in any user-facing file. These are internal development and maintenance tools.

The boundary is enforced by T1.27 (THINK Master): user-facing files must never reference ops equivalents — not in output, not in documentation, not in internal code. Redundant code is acceptable to enforce this boundary.

User testing: Users are given `ais_bench` for quality validation — running benchmark questions against a corpus. They are never given a raw test runner. Integration tests (corpus lifecycle, upload validation, API contract) are operator tools.

Naming Convention Exceptions

The `_ops` suffix rule has deliberate exceptions — commands that are operator-only but predate or are exempt from the suffix convention:

Command	Reason for exception
`ais_deploy`	Core operator infrastructure — predates `_ops` convention; renaming would break every operator workflow

Command	Reason for exception
`ais_commit`	Same as `ais_deploy`
`ais_packet`	Same — EOS toolchain
`ais_bundle`	Same — EOS toolchain
`ais_publish`	Same — EOS toolchain
`ais_backup`	Same — predates convention
`ais_test_ops`	Follows convention — operator integration test runner

These exceptions are documented in `STD - General - Naming Conventions`. The renaming refactor is tracked as AIStudio_093 — deferred post-Beta. Until then, treat the above list as the canonical exception register. Any new operator command without `_ops` suffix must be explicitly added to this table and AIStudio_093 scope.

★★★ ★★★